Screenshots & Analysis Question Answers

1a ./test_objects output

```
sohan@Sohan:~/os-u4-orange-problem$ nano object.c
sohan@Sohan:~/os-u4-orange-problem$ make test_objects
gcc -Wall -Wextra -O2 -c object.c -o object.o
object.c: In function ‘object_read’:
object.c:169:5: warning: ignoring return value of ‘fread’ declared with attribute ‘warn_unused_result’ [-Wunused-result]
  169 |     fread(buf, 1, size, fp);
      |     ^~~~~~~~~~~~~~~~~~~~~~~
gcc -o test_objects test_objects.o object.o -lcrypto
sohan@Sohan:~/os-u4-orange-problem$ ./test_objects
Stored blob with hash: d58213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
Object stored at: .pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
PASS: blob storage
PASS: deduplication
test_objects: test_objects.c:80: test_integrity: Assertion `rc == -1' failed.
Aborted (core dumped)
sohan@Sohan:~/os-u4-orange-problem$ nano object.c
sohan@Sohan:~/os-u4-orange-problem$ make test_objects
gcc -Wall -Wextra -O2 -c object.c -o object.o
object.c: In function ‘object_read’:
object.c:169:5: warning: ignoring return value of ‘fread’ declared with attribute ‘warn_unused_result’ [-Wunused-result]
  169 |     fread(buf, 1, size, fp);
      |     ^~~~~~~~~~~~~~~~~~~~~~~
gcc -o test_objects test_objects.o object.o -lcrypto
sohan@Sohan:~/os-u4-orange-problem$ ./test_objects
Stored blob with hash: d58213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
Object stored at: .pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
PASS: blob storage
PASS: deduplication
PASS: integrity check

All Phase 1 tests passed.
```

1b find .pes/objects -type f

```
All Phase 1 tests passed.
sohan@Sohan:~/os-u4-orange-problem$ find .pes/objects -type f
.pes/objects/25/ef1fa07ea68a52f800dc80756ee6b7ae34b337afedb9b46a1af8e11ec4f476
.pes/objects/2a/594d39232787fba8eb7287418aec99c8fc2ecdaf5aaf2e650eda471e566fcf
.pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
```

2a ./test_tree output

```
sohan@Sohan:~/os-u4-orange-problem$ ./test_tree
Serialized tree: 139 bytes
PASS: tree serialize/parse roundtrip
PASS: tree deterministic serialization

All Phase 2 tests passed.
sohan@Sohan:~/os-u4-orange-problem$
```

2b Tree object structure

```
sohan@Sohan:~/os-u4-orange-problem$ find .pes
.pes
.pes/refs
.pes/refs/heads
.pes/objects
.pes/objects/9f
.pes/objects/9f/046754297947e3cb0b5b18814ee2cc10fd5278e50a23cd43801562774f43bc
.pes/objects/e3
.pes/objects/e3/631e68b75accaff2a3c545fb8877aa20bba6c56180364e61f977365d3bff0f
```

3a pes init → pes add → pes status

```
make: *** [Makefile:14: index.o] Error 1
sohan@Sohan:~/os-u4-orange-problem$ rm index.c
sohan@Sohan:~/os-u4-orange-problem$ nano index.c
sohan@Sohan:~/os-u4-orange-problem$ make clean
rm -f pes test_objects test_tree object.o tree.o index.o commit.o pes.o test_objects.o test_tree.o
rm -rf .pes
sohan@Sohan:~/os-u4-orange-problem$ make
gcc -Wall -Wextra -O2 -c object.c -o object.o
gcc -Wall -Wextra -O2 -c tree.c -o tree.o
gcc -Wall -Wextra -O2 -c index.c -o index.o
gcc -Wall -Wextra -O2 -c commit.c -o commit.o
gcc -Wall -Wextra -O2 -c pes.c -o pes.o
gcc -o pes object.o tree.o index.o commit.o pes.o -lcrypto
sohan@Sohan:~/os-u4-orange-problem$ ./pes init
echo "hello world" > file1.txt
./pes add file1.txt
./pes status
cat .pes/index
Initialized empty PES repository in .pes/
Staged changes:
 staged: file1.txt
100644 0000000000000000000000000000000000000000000000000000000000000000 1776253618 12 file1.txt
sohan@Sohan:~/os-u4-orange-problem$
```

3b cat .pes/index

```
sohan@Sohan:~/os-u4-orange-problem$ ./pes add file1.txt
cat .pes/index
100644 0000000000000000000000000000000000000000000000000000000000000000 1776253618 12 file1.txt
sohan@Sohan:~/os-u4-orange-problem$
```

4a Commit creation

```
sohan@Sohan:~/os-u4-orange-problem$ echo "one" >> file1.txt
./pes add file1.txt
./pes commit -m "Second commit"

echo "two" >> file1.txt
./pes add file1.txt
./pes commit -m "Third commit"

./pes log
Committed: 97ca58b0ab0b... Second commit
Committed: 1ee14314cc0b... Third commit
No commits yet.
sohan@Sohan:~/os-u4-orange-problem$ echo "one" >> file1.txt
sohan@Sohan:~/os-u4-orange-problem$ ./pes add file1.txt
sohan@Sohan:~/os-u4-orange-problem$ ./pes commit -m "Second commit"
Committed: a0ef39a5c409... Second commit
sohan@Sohan:~/os-u4-orange-problem$ echo "two" >> file1.txt
sohan@Sohan:~/os-u4-orange-problem$ ./pes add file1.txt
sohan@Sohan:~/os-u4-orange-problem$ ./pes commit -m "Third commit"
Committed: a6578fa399d1... Third commit
```

4b Object growth

```
sohan@Sohan:~/os-u4-orange-problem$ find .pes -type f | sort
.pes/index
.pes/objects/1e/e14314cc0bf1d8d447378f07a60d64406e12a1e6f069852026dea5fba47e77
.pes/objects/97/ca58b0ab0ba3837433ce361d5c14ebc427d630e8bc1e33d078f0a6e064e9d3
.pes/objects/9f/046754297947e3cb0b5b18814ee2cc10fd5278e50a23cd43801562774f43bc
.pes/objects/a0/ef39a5c4093e137776ecfe315b35fc29559b51deedad65607cffe1a08ad083
.pes/objects/a6/578fa399d1a00b650418c4e5ed170d8cc7bd80f1cb356dd6303301e9800c47
.pes/objects/e3/631e68b75accaff2a3c545fb8877aa20bba6c56180364e61f977365d3bff0f
```

4c Repository refs structure

```
sohan@Sohan:~/os-u4-orange-problem$ find .pes
.pes
.pes/refs
.pes/refs/heads
.pes/index
.pes/objects
.pes/objects/a0
.pes/objects/a0/ef39a5c4093e137776ecfe315b35fc29559b51deedad65607cffe1a08ad083
.pes/objects/1e
.pes/objects/1e/e14314cc0bf1d8d447378f07a60d64406e12a1e6f069852026dea5fba47e77
.pes/objects/9f
.pes/objects/9f/046754297947e3cb0b5b18814ee2cc10fd5278e50a23cd43801562774f43bc
.pes/objects/e3
.pes/objects/e3/631e68b75accaff2a3c545fb8877aa20bba6c56180364e61f977365d3bff0f
.pes/objects/97
.pes/objects/97/ca58b0ab0ba3837433ce361d5c14ebc427d630e8bc1e33d078f0a6e064e9d3
.pes/objects/a6
.pes/objects/a6/578fa399d1a00b650418c4e5ed170d8cc7bd80f1cb356dd6303301e9800c47
```

Q5.1 Branching:
Branching allows creating separate lines of development. Each branch points to a different commit and lets work continue independently.

Q5.2 Why branching is useful:
Branching helps develop features or bug fixes without affecting stable code. Later branches can be merged safely.

Q5.3 Branching in PES:
In PES, branching can be implemented using separate references inside `.pes/refs/heads`, where each branch stores the latest commit hash.

Q6.1 Garbage Collection:
Garbage collection removes unreachable objects from the object store when they are no longer referenced by commits or trees.

Q6.2 Why GC is needed:
GC saves storage space by deleting unused blobs, trees, and commits.